# MERISE — Hôtel Clair de Lune

**Auteur :** Julien Lemarchand
**Créé le :** 2026-03-17
**Dernière mise à jour :** 2026-03-17
**Décisions validées le :** 2026-03-17

### Cible technique

**SGBD cible : PostgreSQL** (via Drizzle ORM + `drizzle-orm/pg-core`). Les types physiques (SQL) n'apparaissent qu'au niveau MPD (section 3).

### Convention de nommage

Ce document utilise un **nommage bilingue** :
les descriptions et titres de sections sont en français, mais tous les noms de tables et d'attributs apparaissent en **anglais** — identiques à ce qui sera dans le code.

| Français (doc) | Anglais (code) |
|---|---|
| Utilisateur | `user` |
| Établissement | `establishment` |
| Suite | `suite` |
| Image | `image` |
| Aménité | `amenity` |
| Option | `option` |
| Réservation | `booking` |
| Avis | `review` |
| Demande de renseignement | `inquiry` |

---

## 1. MCD — Modèle Conceptuel de Données

> Le MCD décrit le domaine métier de manière **indépendante de toute technologie**. Il utilise le formalisme Entité-Association : des **entités** (objets métier), des **propriétés** (caractéristiques), des **associations** (liens entre entités) et des **cardinalités** (règles de participation).
>
> **Ce qui n'apparaît PAS au niveau MCD :** clés étrangères, types SQL, contraintes physiques (NOT NULL, CHECK, DEFAULT), indexes, tables de jointure. Ces concepts relèvent du MLD ou du MPD.

### 1.1 Dictionnaire de données

> Les types ci-dessous sont des **types conceptuels** (texte, numérique, etc.), pas des types SQL. Les types physiques PostgreSQL seront formalisés au MPD (section 3). Les clés étrangères n'apparaissent pas ici

> — les liens entre entités sont exprimés par les associations (section 1.3).

**UTILISATEUR (`user`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant (géré par Better Auth) |
| `name` | texte | Oui | Nom complet |
| `email` | texte | Oui | Adresse e-mail (unique) |
| `email_verified` | booléen | Oui | E-mail vérifié (booléen imposé par Better Auth — un horodatage nullable serait préférable en conception libre, pour savoir *quand* l'e-mail a été vérifié) |
| `image` | texte | Non | Avatar / photo de profil |
| `role` | texte | Oui | Rôle : `client`, `manager`, `admin` |
| `created_at` | horodatage | Oui | Date de création |
| `updated_at` | horodatage | Oui | Date de dernière modification |
| `deleted_at` | horodatage | Non | Soft delete (null = actif) |

> **Note :** Les tables `session`, `account` et `verification` de Better Auth ne sont pas représentées dans le MCD car elles relèvent de l'infrastructure d'authentification, pas du domaine métier.

**ÉTABLISSEMENT (`establishment`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `name` | texte | Oui | Nom de l'établissement |
| `address` | texte | Oui | Adresse postale |
| `postal_code` | texte | Oui | Code postal |
| `city` | texte | Oui | Ville |
| `description` | texte | Non | Description de l'établissement |
| `image` | texte | Non | Image principale (URL) |
| `phone` | texte | Non | Numéro de téléphone |
| `email` | texte | Non | E-mail de contact |
| `check_in_time` | heure | Oui | Heure d'arrivée (ex: 15:00) |
| `check_out_time` | heure | Oui | Heure de départ (ex: 11:00) |
| `created_at` | horodatage | Oui | Date de création |

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `updated_at` | horodatage | Oui | Date de dernière modification |
| `deleted_at` | horodatage | Non | Soft delete (null = actif) |

**SUITE (`suite`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `title` | texte | Oui | Nom de la suite |
| `description` | texte | Non | Description détaillée |
| `price` | décimal | Oui | Prix par nuit (fixe) |
| `main_image` | texte | Oui | URL de l'image principale |
| `capacity` | entier | Oui | Nombre de personnes max |
| `area` | décimal | Non | Surface en $m^2$ |
| `created_at` | horodatage | Oui | Date de création |
| `updated_at` | horodatage | Oui | Date de dernière modification |
| `deleted_at` | horodatage | Non | Soft delete (null = actif) |

**IMAGE (`image`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `url` | texte | Oui | URL de l'image |
| `alt` | texte | Non | Texte alternatif (accessibilité) |
| `position` | entier | Oui | Ordre dans la galerie (unique par suite) |
| `created_at` | horodatage | Oui | Date de création |

**AMÉNITÉ (`amenity`)**

Caractéristique d'un établissement ou d'une suite (WiFi, piscine, douche, PMR...). Pas de prix associé — c'est un attribut booléen "on l'a ou on ne l'a pas".

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `name` | texte | Oui | Libellé (ex: "WiFi gratuit") |
| `slug` | texte | Oui | Identifiant URL/i18n (ex: `wifi-gratuit`) |

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `category` | texte | Oui | Catégorie d'affichage (ex: "Salle de bain", "Technologie") |
| `scope` | texte | Oui | `property`, `room` ou `both` |
| `icon` | texte | Non | Identifiant d'icône UI |

> **Règle de cascade :** si une aménité est cochée au niveau établissement, elle s'applique automatiquement à toutes ses suites et ne peut pas être décochée suite par suite. En base, on ne duplique pas : on stocke le lien au niveau établissement, et à l'affichage on fusionne avec les aménités supplémentaires propres à la suite.

**OPTION (`option`)**

Service proposé par un établissement (petit-déjeuner, lit supplémentaire, parking...). A un prix et un modèle de tarification. Peut être inclus automatiquement ou sélectionné par le client.

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `name` | texte | Oui | Libellé (ex: "Petit-déjeuner") |
| `slug` | texte | Oui | Identifiant URL (ex: `breakfast`) |
| `description` | texte | Non | Description détaillée |
| `icon` | texte | Non | Identifiant d'icône UI |
| `pricing_model` | texte | Oui | Modèle de tarification par défaut |
| `default_price` | décimal | Oui | Prix par défaut |

**Modèles de tarification (`pricing_model`) :**

| Valeur | Signification | Exemple |
|---|---|---|
| `per_person_per_night` | Par personne et par nuit | Petit-déjeuner, demi-pension |
| `per_night` | Par nuit | Lit supplémentaire, parking, animal |
| `per_person_per_stay` | Par personne et par séjour | Accès spa |
| `per_stay` | Forfait par séjour | Pack romantique |
| `per_unit` | À l'unité | Panier pique-nique, vélos |

**RÉSERVATION (`booking`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `reference` | texte | Oui | Référence lisible unique (ex: CDL-2026-0042) |
| `check_in` | date | Oui | Date d'arrivée |
| `check_out` | date | Oui | Date de départ |
| `guest_count` | entier | Oui | Nombre de personnes |
| `price_per_night` | décimal | Oui | Snapshot du prix/nuit au moment de la réservation |
| `total_price` | décimal | Oui | Prix total facturé, figé (hébergement + options) |
| `status` | texte | Oui | `pending`, `confirmed`, `cancelled`, `completed` |
| `cancelled_at` | horodatage | Non | Date d'annulation (si applicable) |
| `created_at` | horodatage | Oui | Date de création de la réservation |
| `updated_at` | horodatage | Oui | Date de dernière modification |

**AVIS (`review`)**

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |
| `rating` | entier | Oui | Note de 1 à 5 |
| `comment` | texte | Non | Commentaire textuel |
| `flagged` | booléen | Oui | Signalé par un gérant (défaut: false) |
| `created_at` | horodatage | Oui | Date de publication |
| `updated_at` | horodatage | Oui | Date de dernière modification |

> **RGPD :** Un client peut demander la suppression de ses données. L'avis est alors **anonymisé** (lien vers l'utilisateur supprimé), pas supprimé. Le contenu reste visible. **Modération :** Un gérant peut **signaler** un avis (`flagged = true`) mais ne peut pas le supprimer. La suppression d'avis négatifs authentiques est interdite (article L121-1 Code de la consommation).

**DEMANDE DE RENSEIGNEMENT (`inquiry`)**

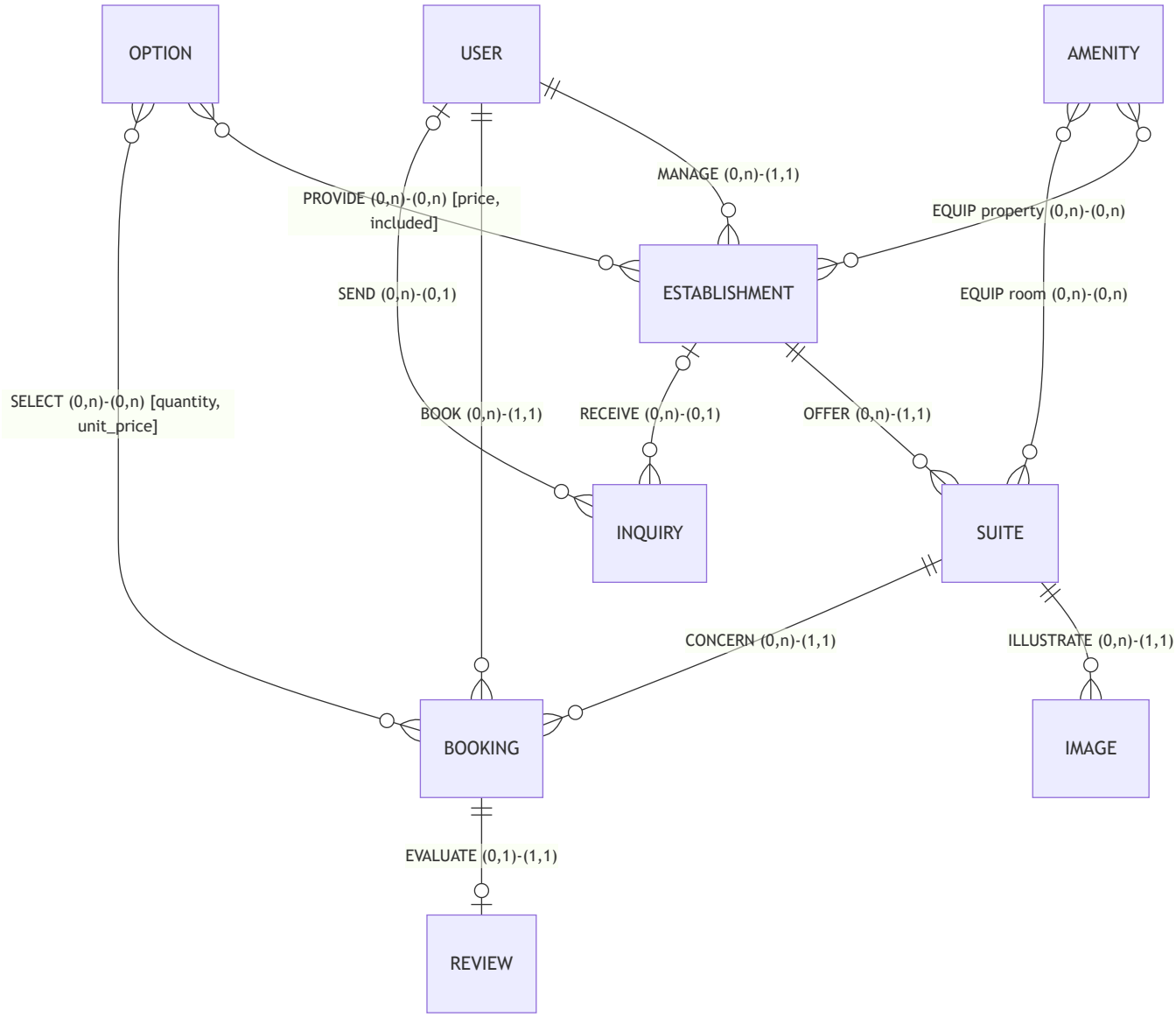
Message de prise de contact soumis via le formulaire du site. Ce n'est pas un thread conversationnel : c'est un message one-shot. La réponse du gérant/admin est envoyée par email (via Resend) et le statut passe à `replied`.

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `id` | texte | Oui | Identifiant |

| Propriété | Type conceptuel | Obligatoire | Description |
|---|---|---|---|
| `name` | texte | Oui | Nom de l'expéditeur |
| `email` | texte | Oui | E-mail de l'expéditeur |
| `subject` | texte | Oui | Sujet prédéfini (enum) |
| `message` | texte | Oui | Corps du message |
| `status` | texte | Oui | `unread`, `read`, `replied` |
| `created_at` | horodatage | Oui | Date d'envoi |

## 1.2 Diagramme entité-association (MCD)

> Diagramme entité-association au format Mermaid ERD (notation Crow's Foot pour les cardinalités). Conformément au formalisme MCD, ce diagramme ne montre **ni clés étrangères, ni types, ni contraintes**. Les propriétés de chaque entité sont détaillées dans le dictionnaire (section 1.1). Les associations porteuses de données (PROVIDE, SELECT) sont documentées en section 1.3.




▶ Source Mermaid

```
erDiagram

    %% ─── Associations 1:N ───

    USER ||--o{ ESTABLISHMENT : "MANAGE (0,n)-(1,1)"
    USER ||--o{ BOOKING : "BOOK (0,n)-(1,1)"
    ESTABLISHMENT ||--o{ SUITE : "OFFER (0,n)-(1,1)"
    SUITE ||--o{ IMAGE : "ILLUSTRATE (0,n)-(1,1)"
    SUITE ||--o{ BOOKING : "CONCERN (0,n)-(1,1)"

    %% ─── Associations N:N ───

    AMENITY }o--o{ ESTABLISHMENT : "EQUIP property (0,n)-(0,n)"
    AMENITY }o--o{ SUITE : "EQUIP room (0,n)-(0,n)"
    OPTION }o--o{ ESTABLISHMENT : "PROVIDE (0,n)-(0,n) [price, included]"
    OPTION }o--o{ BOOKING : "SELECT (0,n)-(0,n) [quantity, unit_price]"

    %% ─── Associations 1:1 et optionnelles ───

    BOOKING ||--o| REVIEW : "EVALUATE (0,1)-(1,1)"
    ESTABLISHMENT |o--o{ INQUIRY : "RECEIVE (0,n)-(0,1)"
    USER |o--o{ INQUIRY : "SEND (0,n)-(0,1)"
```

### 1.3 Détail des associations et cardinalités

| Association | Entité A | Card. A | Entité B | Card. B | Propriétés portées | Description |
|---|---|---|---|---|---|---|
| **MANAGE** | User (manager) | 0,n | Establishment | 1,1 | — | Un gérant gère 0 à N établissements (un user n'est pas forcément gérant). Un établissement a exactement un gérant. |
| **OFFER** | Establishment | 0,n | Suite | 1,1 | — | Un établissement propose 0 à N suites. Une suite appartient à un seul établissement. |
| **ILLUSTRATE** | Suite | 0,n | Image | 1,1 | — | Une suite possède 0 à N images dans sa galerie. |
| **EQUIP (property)** | Establishment | 0,n | Amenity | 0,n | — | Un établissement dispose de 0 à N aménités. |

| Association | Entité A | Card. A | Entité B | Card. B | Propriétés portées | Description |
|---|---|---|---|---|---|---|
| **EQUIP (room)** | Suite | 0,n | Amenity | 0,n | — | Une suite dispose de 0 à N aménités supplémentaires. |
| **PROVIDE** | Establishment | 0,n | Option | 0,n | `price` (décimal), `included` (booléen) | Un établissement propose 0 à N options, chacune avec un prix spécifique. `included = true` : option automatiquement ajoutée (affichée "incluse", `unit_price = 0` en réservation). |
| **BOOK** | User (client) | 0,n | Booking | 1,1 | — | Un client effectue 0 à N réservations. |
| **SELECT** | Booking | 0,n | Option | 0,n | `quantity` (entier), `unit_price` (décimal) | Une réservation inclut 0 à N options, avec quantité et prix snapshot. |
| **CONCERN** | Suite | 0,n | Booking | 1,1 | — | Une suite est concernée par 0 à N réservations. |
| **EVALUATE** | Booking | 0,1 | Review | 1,1 | — | Une réservation peut avoir 0 ou 1 avis. |
| **RECEIVE** | Establishment | 0,n | Inquiry | 0,1 | — | Un établissement reçoit 0 à N messages (nullable si sujet technique). |
| **SEND** | User | 0,n | Inquiry | 0,1 | — | Un user connecté peut envoyer 0 à N messages (nullable si visiteur anonyme). |

### 1.4 Règles de gestion

1. **Rôles :** Un utilisateur a un seul rôle (`admin`, `manager`, `client`). Le visiteur n'est pas un utilisateur (pas de compte).
2. **Gérant → Établissement(s) :** Relation 1:N. Un gérant peut gérer plusieurs établissements (ex: hôtels proches géographiquement). Un établissement a exactement un gérant.
3. **Check-in / Check-out :** Chaque établissement définit ses horaires d'arrivée et de départ. Purement informatif, pas de logique de disponibilité horaire.

4. **Prix fixe :** Le prix d'une suite est fixe quelle que soit la période. Au moment de la réservation, le prix est copié (`price_per_night`) pour garantir l'intégrité historique.
5. **Disponibilité :** Une suite ne peut pas être réservée deux fois sur des dates qui se chevauchent (contrôle : `new_check_in < existing_check_out AND new_check_out > existing_check_in`).
6. **Parcours de réservation :** La réservation est créée au statut `pending` (snapshot des prix effectué). Le client passe par un écran de paiement (mocké — validation automatique). Le paiement réussi fait passer le statut à `confirmed`. Un abandon ou timeout laisse la réservation en `pending` (nettoyable par cron).
7. **Annulation :** Possible uniquement depuis le statut `confirmed` et si la date d'arrivée est dans plus de 3 jours. Le statut passe à `cancelled`, la suite redevient disponible.
8. **Suppression douce :** Les établissements, suites et utilisateurs ne sont jamais supprimés physiquement. Le champ `deleted_at` est renseigné.
9. **Aménités — cascade :** Si une aménité est cochée au niveau établissement, elle s'applique à toutes ses suites automatiquement et ne peut pas être décochée suite par suite. Les suites peuvent avoir des aménités supplémentaires.
10. **Options — inclusion :** Un établissement peut marquer une option comme `included = true` (ex: petit-déjeuner inclus). Une option incluse est **automatiquement ajoutée** à toute réservation de cet établissement — le client ne peut pas la retirer. Son prix reste traçable dans `establishment_option.price` (utile pour la gestion/comptabilité), mais côté client elle est affichée comme "incluse" dans le prix de la suite. Le `unit_price` snapshot dans `booking_option` est à `0` pour les options incluses (pas de surcoût facturé).
11. **Options — snapshot prix :** Au moment de la réservation, les prix des options sont copiés dans `booking_option.unit_price` pour garantir l'intégrité historique. Pour les options incluses, `unit_price = 0` (le coût réel reste dans `establishment_option.price` à des fins de gestion).
12. **Prix total :** `total_price = subtotal + options_total`, où `subtotal = nb_nuits × price_per_night` et `options_total = somme des (quantity × unit_price)`. Les options incluses ayant `unit_price = 0`, elles apparaissent dans la réservation (traçabilité) sans impacter le montant facturé.
13. **Review :** Un avis ne peut être laissé que sur une réservation au statut `completed`, et un seul avis par réservation. Un gérant peut signaler un avis (`flagged = true`) mais ne peut pas le supprimer (article L121-1 Code de la consommation).
14. **Review — RGPD :** En cas de demande de suppression de données, l'avis est anonymisé (lien vers l'utilisateur supprimé), pas supprimé.
15. **Inquiry :** Message one-shot via formulaire. Les sujets sont prédéfinis (`complaint`, `extra_service`, `suite_info`, `app_issue`). Le lien vers l'établissement est optionnel (null pour les sujets techniques, routés vers l'admin). Le lien vers l'utilisateur est optionnel (null pour les visiteurs anonymes). La réponse est envoyée par email via Resend, pas de thread in-app.

---

## 2. MLD — Modèle Logique de Données

> Le MLD traduit le MCD en **modèle relationnel**. Chaque entité devient une **relation** (table), les associations 1:N produisent des **clés étrangères**, et les associations N:N produisent des **tables de jointure**.
>
> **Ce qui apparaît au MLD :** relations, attributs, clés primaires (PK), clés étrangères (FK).
> **Ce qui n'apparaît PAS :** types SQL, contraintes physiques (NOT NULL, CHECK, DEFAULT, ON DELETE),

> indexes. Ces éléments relèvent du MPD (section 3).

## 2.1 Règles de passage MCD → MLD appliquées

| Règle | Application dans notre modèle |
|---|---|
| Entité → Relation | Chaque entité du MCD devient une relation |
| Identifiant → Clé primaire | L'identifiant de chaque entité devient la PK |
| Association 1:N → FK | La clé étrangère est placée côté "N" (ex: `establishment.#manager_id`) |
| Association N:N → Table de jointure | PK composite formée des FK des deux entités liées |
| Association N:N avec propriétés → Table de jointure enrichie | Les propriétés de l'association deviennent des attributs de la table de jointure |
| Association 1:1 → FK + contrainte UNIQUE | EVALUATE → `review.#booking_id` (unique) |

## 2.2 Schéma relationnel

> Notation standard MERISE : `_souligné_` = clé primaire, `#préfixé` = clé étrangère. Pas de types ni de contraintes physiques à ce niveau.

**Relations issues de Better Auth (infrastructure — déjà en place)**

- `session (_id_, expires_at, token, created_at, updated_at, ip_address, user_agent, #user_id)`
- `account (_id_, account_id, provider_id, #user_id, access_token, refresh_token, id_token, access_token_expires_at, refresh_token_expires_at, scope, password, created_at, updated_at)`
- `verification (_id_, identifier, value, expires_at, created_at, updated_at)`

> Ces relations ne sont pas modifiées. Elles sont gérées par Better Auth.

**Relations métier**

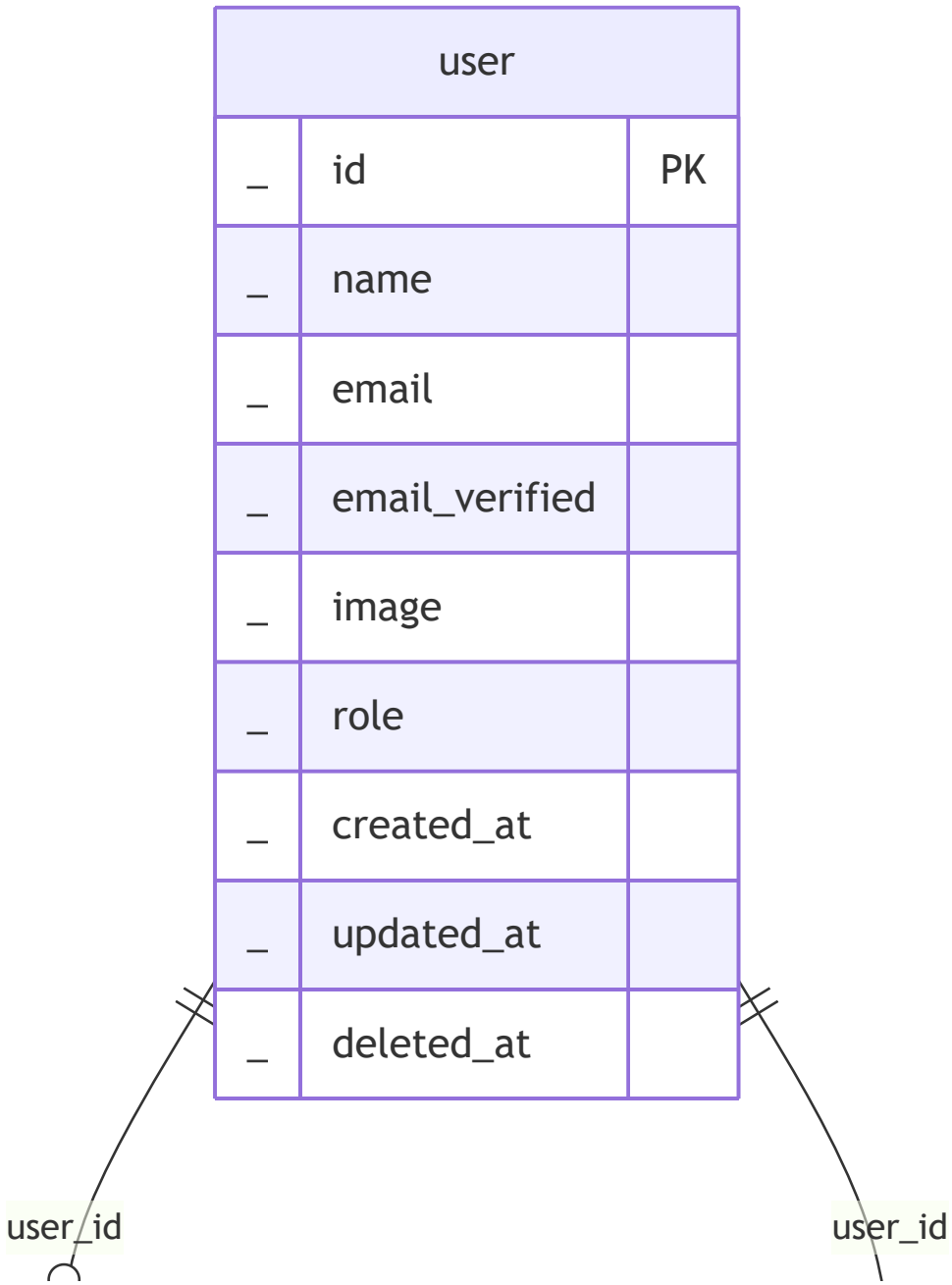
- `user (_id_, name, email, email_verified, image, role, created_at, updated_at, deleted_at)`
- `establishment (_id_, name, address, postal_code, city, description, image, phone, email, check_in_time, check_out_time, created_at, updated_at, deleted_at, #manager_id)`
- `suite (_id_, title, description, price, main_image, capacity, area, created_at, updated_at, deleted_at, #establishment_id)`
- `image (_id_, url, alt, position, created_at, #suite_id)`
- `amenity (_id_, name, slug, category, scope, icon)`
- `establishment_amenity (_#establishment_id, #amenity_id_)`
- `suite_amenity (_#suite_id, #amenity_id_)`
- `option (_id_, name, slug, description, icon, pricing_model, default_price)`
- `establishment_option (_#establishment_id, #option_id_, price, included)`

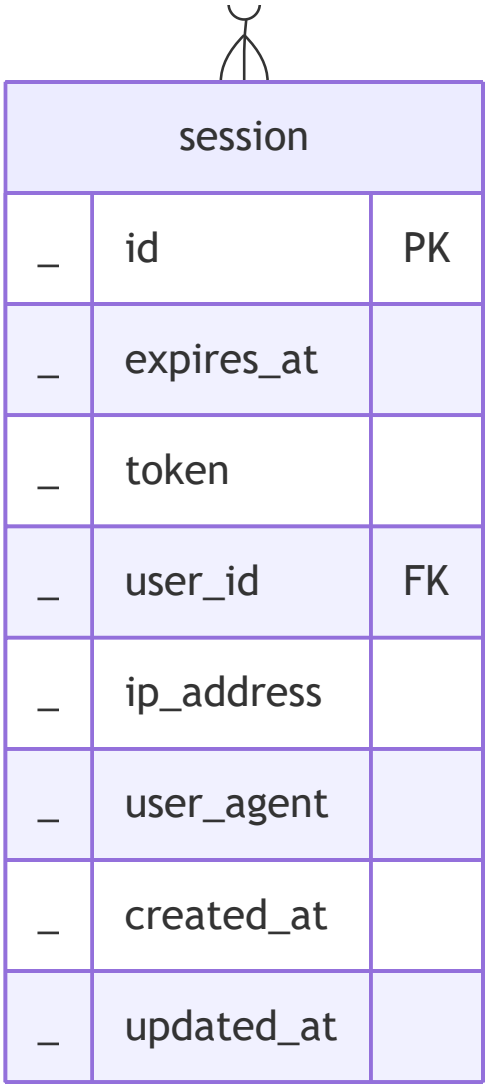
- `booking (_id_, reference, check_in, check_out, guest_count, price_per_night, total_price, status, cancelled_at, created_at, updated_at, #client_id, #suite_id)`
- `booking_option (_#booking_id, #option_id_, quantity, unit_price)`
- `review (_id_, rating, comment, flagged, created_at, updated_at, #booking_id, #user_id)`
- `inquiry (_id_, name, email, subject, message, status, created_at, #establishment_id, #user_id)`

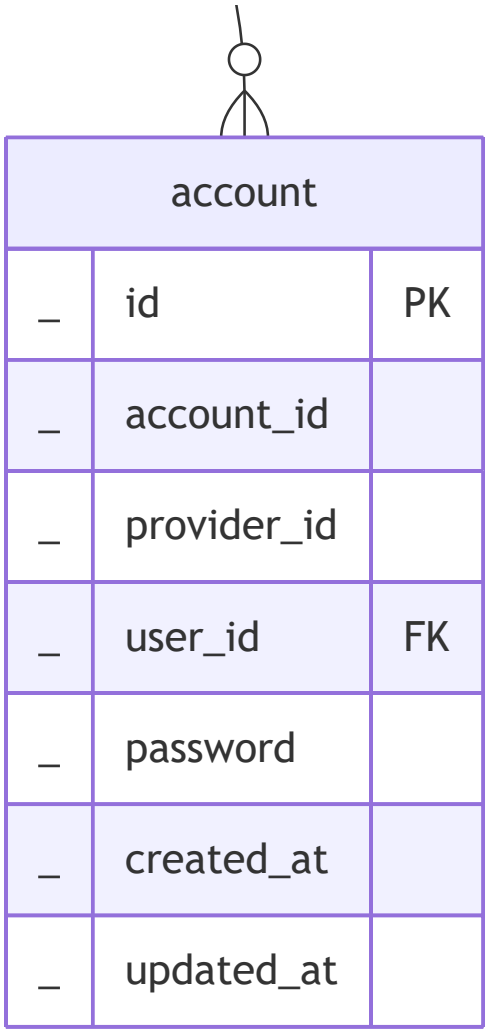
### 2.3 Diagramme relationnel (MLD)

> Ce diagramme Mermaid ERD montre la structure relationnelle issue du passage MCD → MLD. Les relations, clés primaires et clés étrangères sont visibles.
>
> **Note Mermaid :** La syntaxe ERD de Mermaid exige un type pour chaque attribut. Les types indiqués ici sont une contrainte d'outil, pas du formalisme MLD. En MLD strict, seuls les PK et FK sont spécifiés.

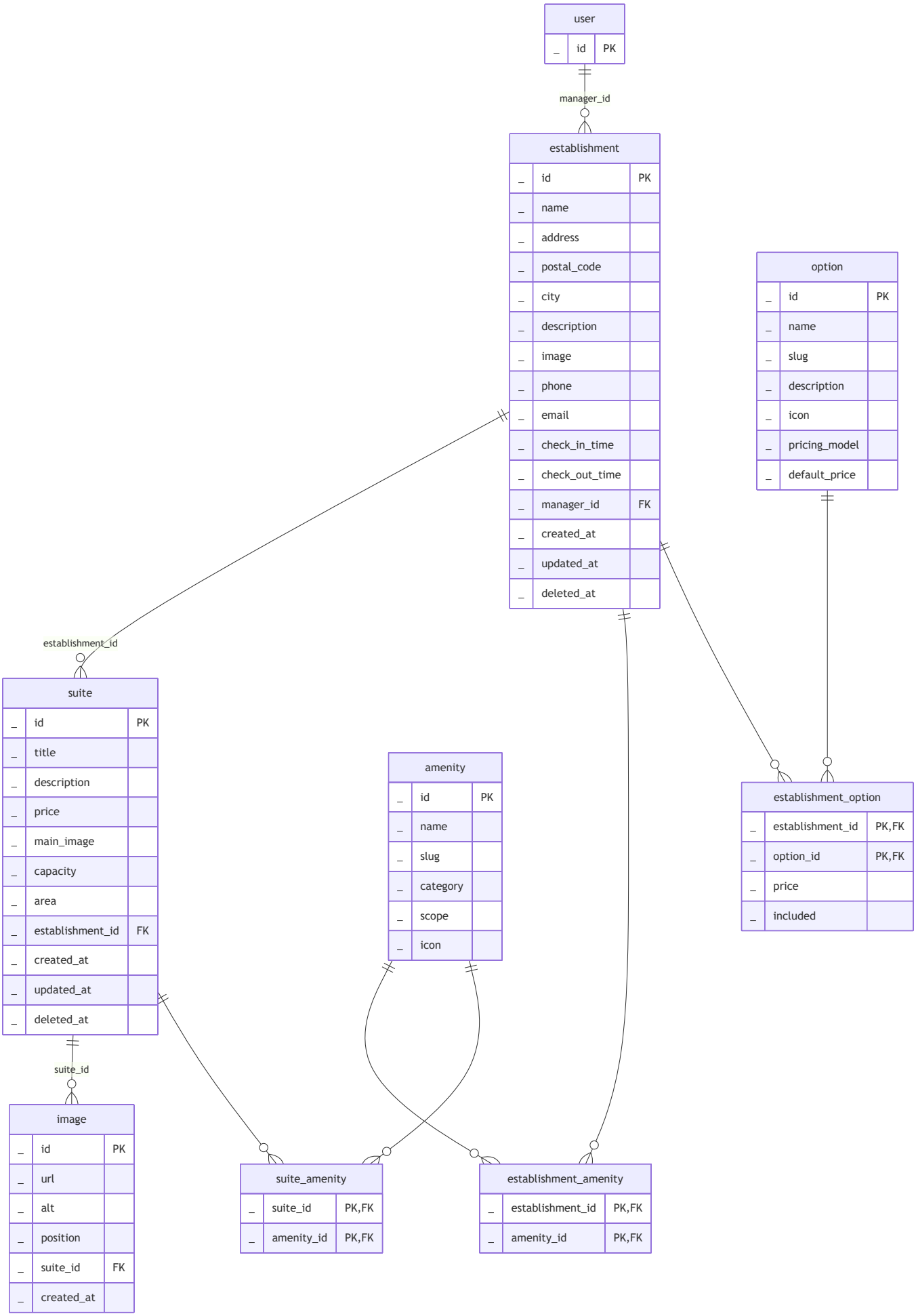
**Auth (user, session, account, verification)**



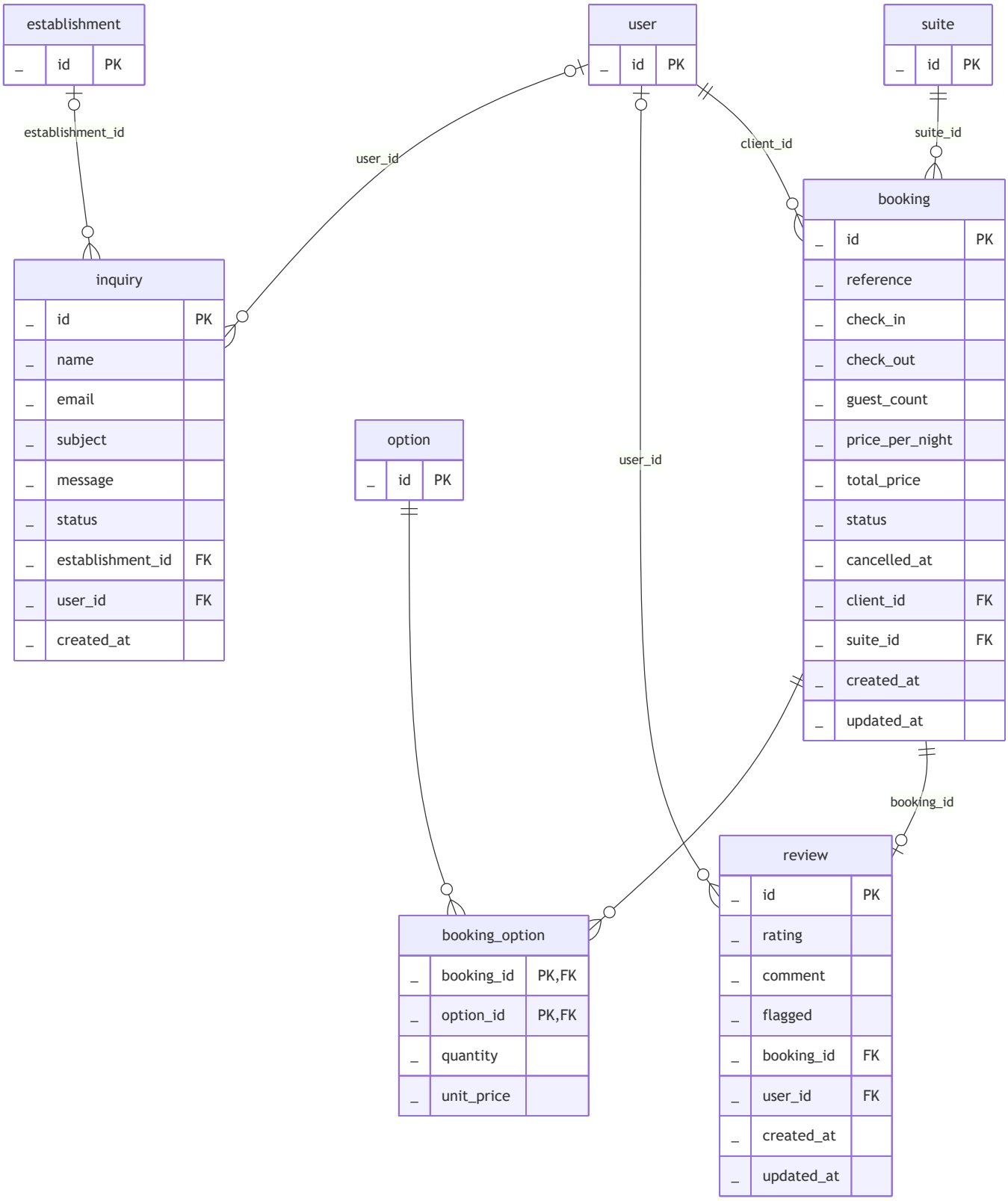



**Catalogue (establishment, suite, image, amenity, option)**




## Transactions (booking, review, inquiry)




▶ Source Mermaid (diagramme complet original)

```
erDiagram
    %% ── Relations issues des associations 1:N ──

    user ||--o{ establishment : "manager_id"
    user ||--o{ booking : "client_id"
    user |o--o{ review : "user_id"
    user |o--o{ inquiry : "user_id"
```

```
    establishment ||--o{ suite : "establishment_id"
    establishment |o--o{ inquiry : "establishment_id"

    suite ||--o{ image : "suite_id"
    suite ||--o{ booking : "suite_id"

    booking ||--o| review : "booking_id"

    %% ─── Tables de jointure (associations N:N) ───

    establishment ||--o{ establishment_amenity : ""
    amenity ||--o{ establishment_amenity : ""

    suite ||--o{ suite_amenity : ""
    amenity ||--o{ suite_amenity : ""

    establishment ||--o{ establishment_option : ""
    option ||--o{ establishment_option : ""

    booking ||--o{ booking_option : ""
    option ||--o{ booking_option : ""

    %% ─── Tables Better Auth ───

    user ||--o{ session : "user_id"
    user ||--o{ account : "user_id"

    session {
        _ id PK
        _ expires_at
        _ token
        _ user_id FK
        _ ip_address
        _ user_agent
        _ created_at
        _ updated_at
    }

    account {
        _ id PK
        _ account_id
        _ provider_id
        _ user_id FK
        _ password
        _ created_at
        _ updated_at
    }

    verification {
        _ id PK
        _ identifier
        _ value
        _ expires_at
```

```
        _ created_at
        _ updated_at
    }

    %% ─── Relations métier ───

    user {
        _ id PK
        _ name
        _ email
        _ email_verified
        _ image
        _ role
        _ created_at
        _ updated_at
        _ deleted_at
    }

    establishment {
        _ id PK
        _ name
        _ address
        _ postal_code
        _ city
        _ description
        _ image
        _ phone
        _ email
        _ check_in_time
        _ check_out_time
        _ manager_id FK
        _ created_at
        _ updated_at
        _ deleted_at
    }

    suite {
        _ id PK
        _ title
        _ description
        _ price
        _ main_image
        _ capacity
        _ area
        _ establishment_id FK
        _ created_at
        _ updated_at
        _ deleted_at
    }

    image {
        _ id PK
        _ url
        _ alt
```

```
        _ position
        _ suite_id FK
        _ created_at
    }

    amenity {
        _ id PK
        _ name
        _ slug
        _ category
        _ scope
        _ icon
    }

    establishment_amenity {
        _ establishment_id PK, FK
        _ amenity_id PK, FK
    }

    suite_amenity {
        _ suite_id PK, FK
        _ amenity_id PK, FK
    }

    option {
        _ id PK
        _ name
        _ slug
        _ description
        _ icon
        _ pricing_model
        _ default_price
    }

    establishment_option {
        _ establishment_id PK, FK
        _ option_id PK, FK
        _ price
        _ included
    }

    booking {
        _ id PK
        _ reference
        _ check_in
        _ check_out
        _ guest_count
        _ price_per_night
        _ total_price
        _ status
        _ cancelled_at
        _ client_id FK
        _ suite_id FK
        _ created_at
```

```
        _ updated_at
    }

    booking_option {
        _ booking_id PK, FK
        _ option_id PK, FK
        _ quantity
        _ unit_price
    }

    review {
        _ id PK
        _ rating
        _ comment
        _ flagged
        _ booking_id FK
        _ user_id FK
        _ created_at
        _ updated_at
    }

    inquiry {
        _ id PK
        _ name
        _ email
        _ subject
        _ message
        _ status
        _ establishment_id FK
        _ user_id FK
        _ created_at
    }
```

---

## 3. MPD — Modèle Physique de Données

> Le MPD enrichit le MLD avec tous les détails nécessaires à l'implémentation physique : **types SQL**, **contraintes** (NOT NULL, UNIQUE, CHECK, DEFAULT), **stratégies ON DELETE**, et spécificités du SGBD cible (PostgreSQL).

### 3.1 Schéma physique détaillé

**Tables issues de Better Auth (infrastructure — déjà en place)**

- `session (id, expires_at, token, created_at, updated_at, ip_address, user_agent, #user_id)`
- `account (id, account_id, provider_id, #user_id, access_token, refresh_token, id_token, access_token_expires_at, refresh_token_expires_at, scope, password, created_at, updated_at)`
- `verification (id, identifier, value, expires_at, created_at, updated_at)`

> Ces tables ne sont pas modifiées. Elles sont gérées par Better Auth.

**Tables métier**

**user** `(id, name, email, email_verified, image, role, created_at, updated_at, deleted_at)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| UNIQUE | `email` |
| CHECK | `role IN ('admin', 'manager', 'client')` |
| DEFAULT | `role = 'client'` |
| NOTE | Table existante Better Auth, enrichie avec `role` + `deleted_at` |

---

**establishment** `(id, name, address, postal_code, city, description, image, phone, email, check_in_time, check_out_time, created_at, updated_at, deleted_at, #manager_id)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| FK | `manager_id → user(id) ON DELETE RESTRICT` |
| NOT NULL | `name, address, postal_code, city, check_in_time, check_out_time, manager_id` |

---

**suite** `(id, title, description, price, main_image, capacity, area, created_at, updated_at, deleted_at, #establishment_id)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| FK | `establishment_id → establishment(id) ON DELETE RESTRICT` |
| NOT NULL | `title, price, main_image, capacity, establishment_id` |
| CHECK | `price > 0, capacity > 0` |

---

**image** `(id, url, alt, position, created_at, #suite_id)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| FK | `suite_id → suite(id) ON DELETE CASCADE` |
| NOT NULL | `url, position, suite_id` |
| UNIQUE | `(suite_id, position)` — pas deux images au même rang dans une suite |

**amenity** (id, name, slug, category, scope, icon)

| Contrainte | Détail |
|---|---|
| PK | id |
| UNIQUE | name, slug |
| NOT NULL | name, slug, category, scope |
| CHECK | scope IN ('property', 'room', 'both') |

**establishment_amenity** (#establishment_id, #amenity_id)

| Contrainte | Détail |
|---|---|
| PK | (establishment_id, amenity_id) |
| FK | establishment_id → establishment(id) ON DELETE CASCADE |
| FK | amenity_id → amenity(id) ON DELETE CASCADE |

**suite_amenity** (#suite_id, #amenity_id)

| Contrainte | Détail |
|---|---|
| PK | (suite_id, amenity_id) |
| FK | suite_id → suite(id) ON DELETE CASCADE |
| FK | amenity_id → amenity(id) ON DELETE CASCADE |

**option** (id, name, slug, description, icon, pricing_model, default_price)

| Contrainte | Détail |
|---|---|
| PK | id |
| UNIQUE | slug |
| NOT NULL | name, slug, pricing_model, default_price |
| CHECK | pricing_model IN ('per_person_per_night', 'per_night', 'per_person_per_stay', 'per_stay', 'per_unit') |

**establishment_option** (#establishment_id, #option_id, price, included)

| Contrainte | Détail |
|---|---|
| PK | (establishment_id, option_id) |
| FK | establishment_id → establishment(id) ON DELETE CASCADE |

| Contrainte | Détail |
|---|---|
| FK | `option_id → option(id) ON DELETE CASCADE` |
| NOT NULL | `price, included` |
| DEFAULT | `included = false` |

**booking** `(id, reference, check_in, check_out, guest_count, price_per_night, total_price, status, cancelled_at, created_at, updated_at, #client_id, #suite_id)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| UNIQUE | `reference` |
| FK | `client_id → user(id) ON DELETE RESTRICT` |
| FK | `suite_id → suite(id) ON DELETE RESTRICT` |
| NOT NULL | `reference, check_in, check_out, guest_count, price_per_night, total_price, status, client_id, suite_id` |
| CHECK | `status IN ('pending', 'confirmed', 'cancelled', 'completed')` |
| DEFAULT | `status = 'pending'` |
| CHECK | `check_out > check_in` |
| CHECK | `price_per_night > 0, total_price > 0, guest_count > 0` |
| CONTRAINTE METIER | Pas de chevauchement de dates pour une même suite |

**booking_option** `(#booking_id, #option_id, quantity, unit_price)`

| Contrainte | Détail |
|---|---|
| PK | `(booking_id, option_id)` |
| FK | `booking_id → booking(id) ON DELETE CASCADE` |
| FK | `option_id → option(id) ON DELETE RESTRICT` |
| NOT NULL | `quantity, unit_price` |
| CHECK | `quantity > 0, unit_price >= 0` |
| NOTE | Total par option = `quantity × unit_price` (calculé à la volée, pas stocké) |

**review** `(id, rating, comment, flagged, created_at, updated_at, #booking_id, #user_id)`

| Contrainte | Détail |
|---|---|

| Contrainte | Détail |
|---|---|
| PK | `id` |
| UNIQUE | `booking_id` — un seul avis par réservation |
| FK | `booking_id → booking(id) ON DELETE CASCADE` |
| FK | `user_id → user(id) ON DELETE SET NULL` — anonymisation RGPD |
| NOT NULL | `rating, booking_id` |
| DEFAULT | `flagged = false` |
| CHECK | `rating BETWEEN 1 AND 5` |
| CONTRAINTE METIER | Réservation au statut `'completed'` uniquement |

---

**inquiry** `(id, name, email, subject, message, status, created_at, #establishment_id, #user_id)`

| Contrainte | Détail |
|---|---|
| PK | `id` |
| FK | `establishment_id → establishment(id) ON DELETE SET NULL` — nullable |
| FK | `user_id → user(id) ON DELETE SET NULL` — nullable |
| NOT NULL | `name, email, subject, message, status` |
| CHECK | `subject IN ('complaint', 'extra_service', 'suite_info', 'app_issue')` |
| CHECK | `status IN ('unread', 'read', 'replied')` |
| DEFAULT | `status = 'unread'` |

### 3.2 Récapitulatif des clés étrangères et ON DELETE

| Table source | Colonne FK | Table cible | Cardinalité | ON DELETE |
|---|---|---|---|---|
| `establishment` | `manager_id` | `user` | N:1 | RESTRICT |
| `suite` | `establishment_id` | `establishment` | N:1 | RESTRICT |
| `image` | `suite_id` | `suite` | N:1 | CASCADE |
| `establishment_amenity` | `establishment_id` | `establishment` | N:N | CASCADE |
| `establishment_amenity` | `amenity_id` | `amenity` | N:N | CASCADE |
| `suite_amenity` | `suite_id` | `suite` | N:N | CASCADE |
| `suite_amenity` | `amenity_id` | `amenity` | N:N | CASCADE |
| `establishment_option` | `establishment_id` | `establishment` | N:N | CASCADE |
| `establishment_option` | `option_id` | `option` | N:N | CASCADE |

| Table source | Colonne FK | Table cible | Cardinalité | ON DELETE |
|---|---|---|---|---|
| `booking` | `client_id` | `user` | N:1 | RESTRICT |
| `booking` | `suite_id` | `suite` | N:1 | RESTRICT |
| `booking_option` | `booking_id` | `booking` | N:N | CASCADE |
| `booking_option` | `option_id` | `option` | N:N | RESTRICT |
| `review` | `booking_id` | `booking` | 1:1 | CASCADE |
| `review` | `user_id` | `user` | N:1 (nullable) | SET NULL |
| `inquiry` | `establishment_id` | `establishment` | N:1 (nullable) | SET NULL |
| `inquiry` | `user_id` | `user` | N:1 (nullable) | SET NULL |

### 3.3 Stratégies ON DELETE

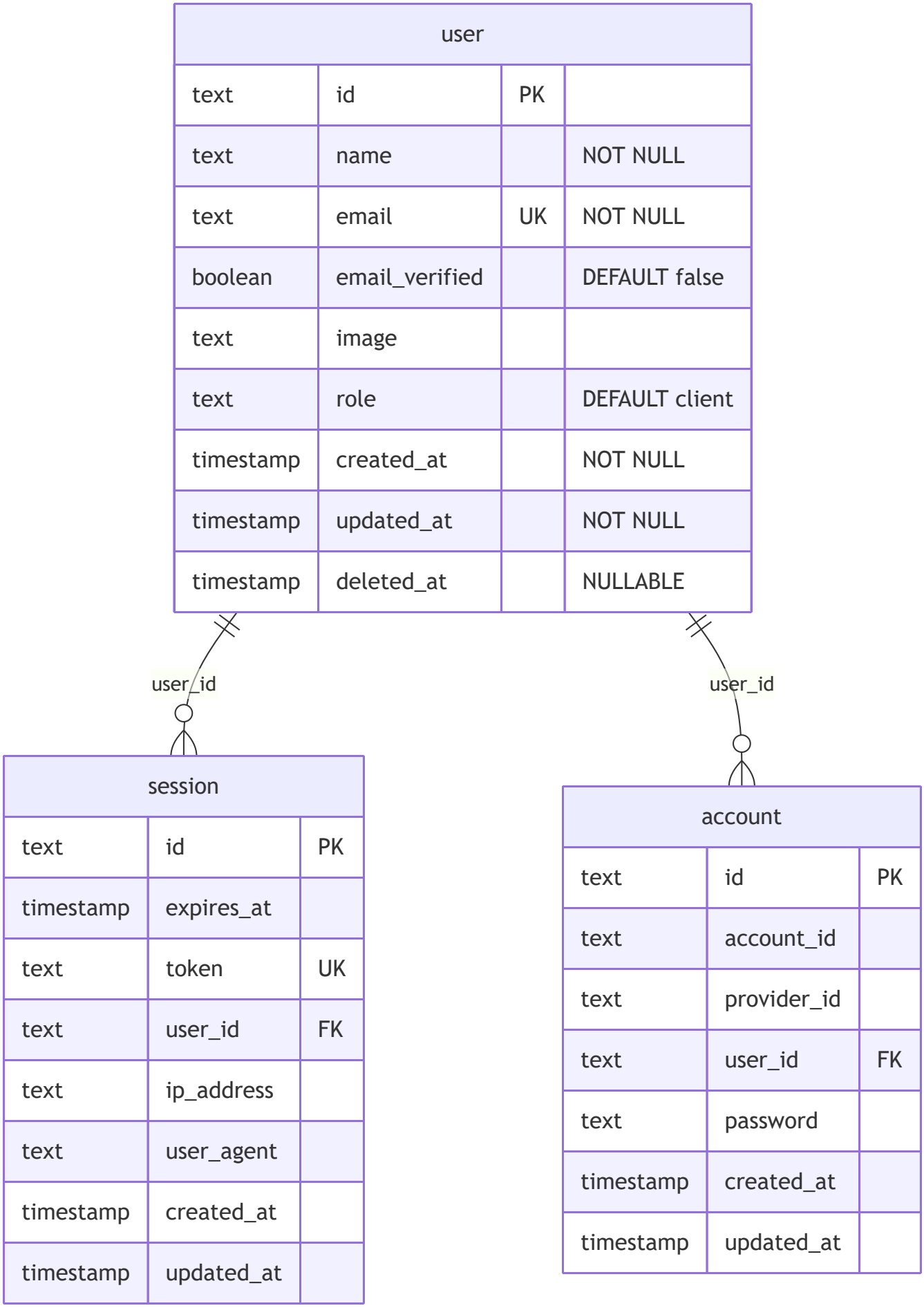
| Stratégie | Appliquée quand | Exemple |
|---|---|---|
| **RESTRICT** | Empêcher la suppression si des données liées existent | Supprimer un user qui a des réservations → bloqué |
| **CASCADE** | La suppression en cascade est logique métier | Supprimer une suite → ses images de galerie et ses amenity links disparaissent |
| **SET NULL** | Le lien peut devenir orphelin sans casser la logique | Supprimer un établissement → les demandes restent (establishment_id = null) |

> **Note :** En pratique, grâce au soft delete (`deleted_at`), les suppressions physiques sont rarissimes. Les contraintes ON DELETE (RESTRICT, SET NULL, CASCADE) servent de **filet de sécurité** au niveau base de données — elles ne se déclenchent que si un DELETE physique est exécuté, ce qui ne devrait arriver qu'en cas de purge administrative ou de conformité RGPD (droit à l'effacement).

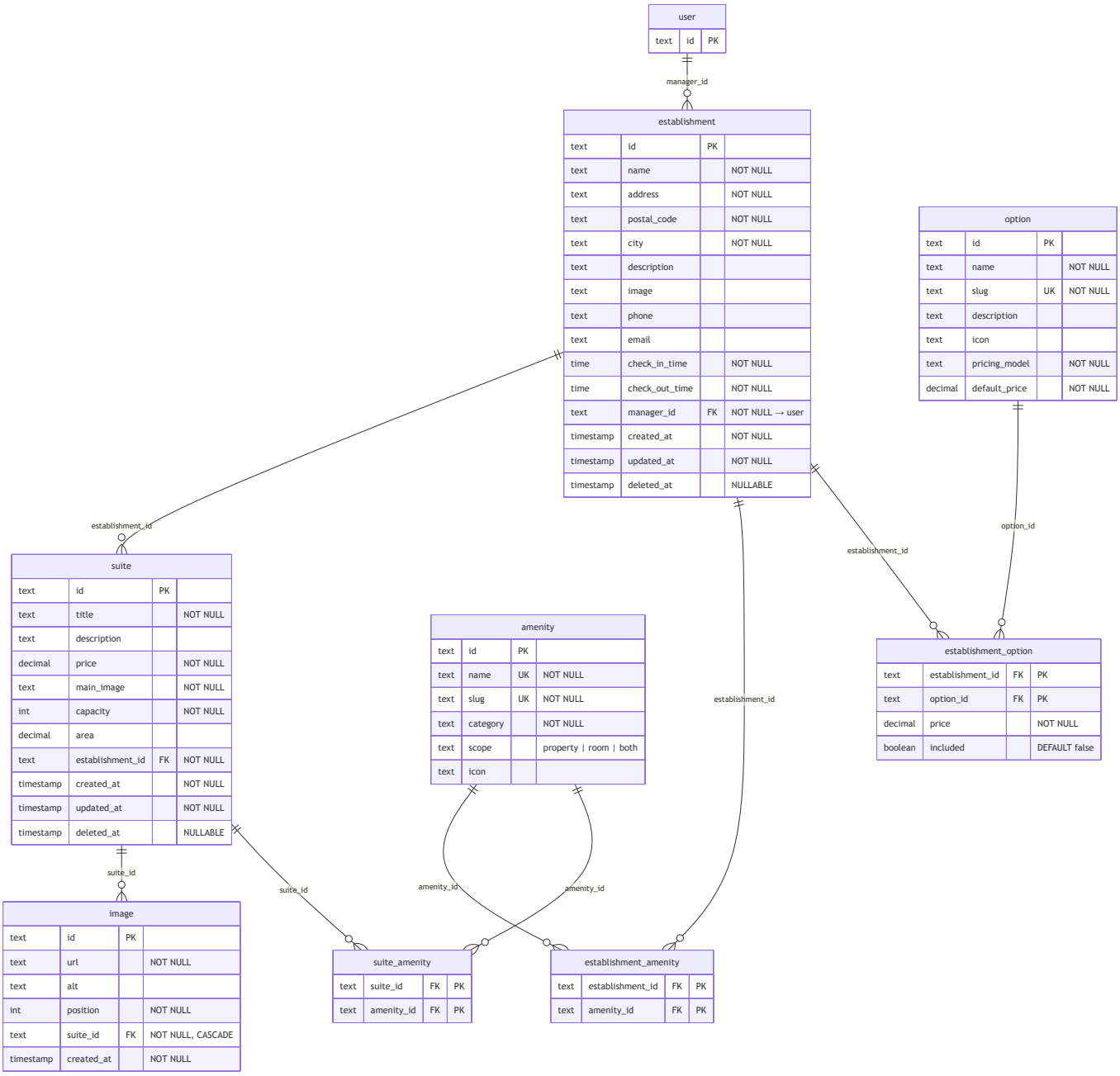
### 3.4 Diagramme physique (MPD)

> Ce diagramme ERD Mermaid représente le MPD complet avec types PostgreSQL, contraintes et annotations physiques.

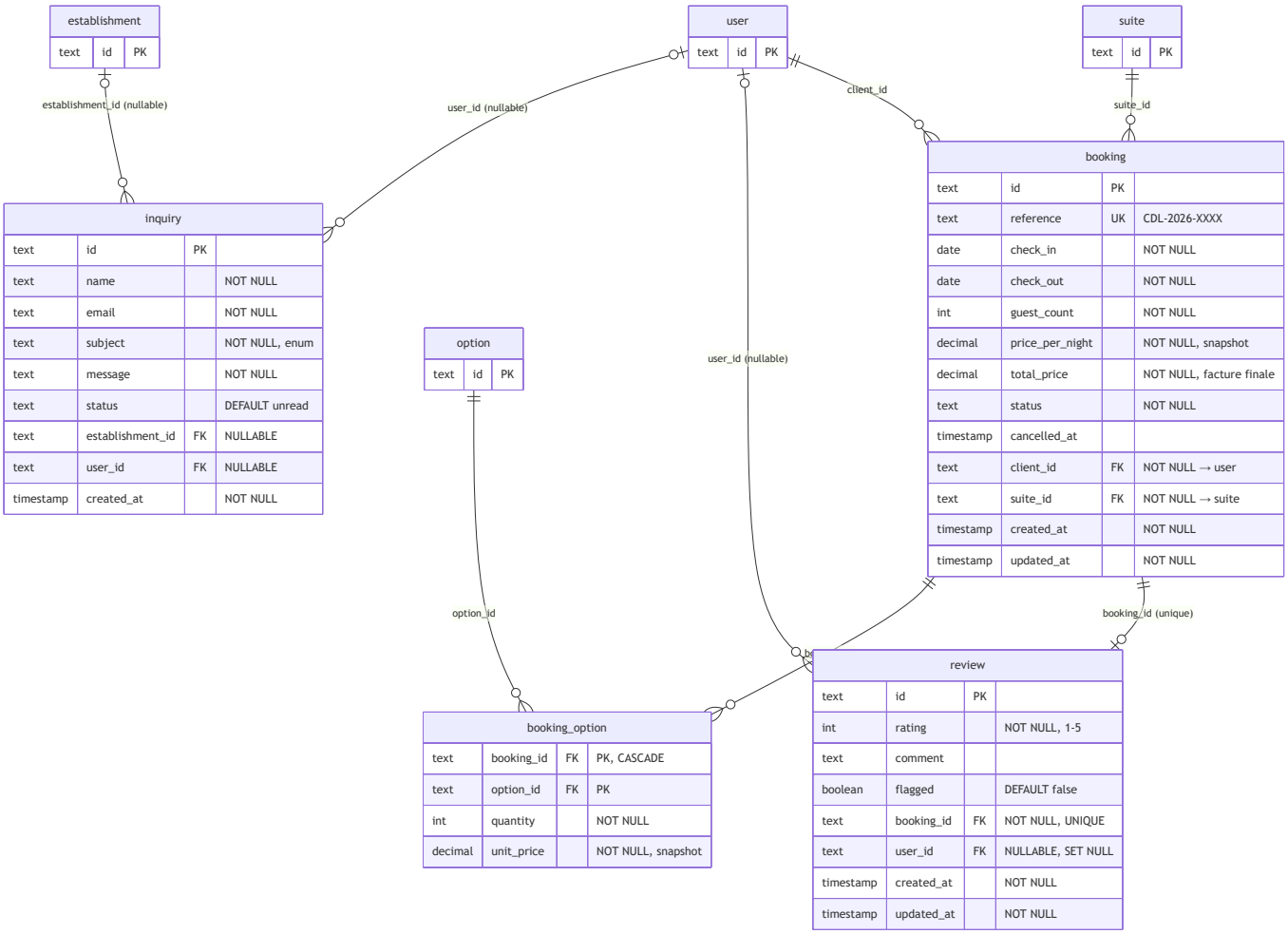
**Auth (user, session, account, verification)**




**Catalogue (establishment, suite, image, amenity, option)**




## Transactions (booking, review, inquiry)




▶ Source Mermaid (diagramme complet original)

```
erDiagram
    %% ── Relations ──

    user ||--o{ establishment : "manager_id"
    user ||--o{ booking : "client_id"
    user |o--o{ review : "user_id (nullable)"
    user |o--o{ inquiry : "user_id (nullable)"

    establishment ||--o{ suite : "establishment_id"
    establishment |o--o{ inquiry : "establishment_id (nullable)"
    establishment ||--o{ establishment_amenity : "establishment_id"
    establishment ||--o{ establishment_option : "establishment_id"

    suite ||--o{ image : "suite_id"
    suite ||--o{ suite_amenity : "suite_id"
    suite ||--o{ booking : "suite_id"

    amenity ||--o{ establishment_amenity : "amenity_id"
    amenity ||--o{ suite_amenity : "amenity_id"

    option ||--o{ establishment_option : "option_id"
    option ||--o{ booking_option : "option_id"

    booking ||--o| review : "booking_id (unique)"
```

```
    booking ||--o{ booking_option : "booking_id"

    %% ─── Tables Better Auth (infrastructure) ───

    user ||--o{ session : "user_id"
    user ||--o{ account : "user_id"

    session {
        text id PK
        timestamp expires_at
        text token UK
        text user_id FK
        text ip_address
        text user_agent
        timestamp created_at
        timestamp updated_at
    }

    account {
        text id PK
        text account_id
        text provider_id
        text user_id FK
        text password
        timestamp created_at
        timestamp updated_at
    }

    verification {
        text id PK
        text identifier
        text value
        timestamp expires_at
        timestamp created_at
        timestamp updated_at
    }

    %% ─── Tables métier ───

    user {
        text id PK
        text name "NOT NULL"
        text email UK "NOT NULL"
        boolean email_verified "DEFAULT false"
        text image
        text role "DEFAULT client"
        timestamp created_at "NOT NULL"
        timestamp updated_at "NOT NULL"
        timestamp deleted_at "NULLABLE"
    }

    establishment {
        text id PK
        text name "NOT NULL"
```

```
        text address "NOT NULL"
        text postal_code "NOT NULL"
        text city "NOT NULL"
        text description
        text image
        text phone
        text email
        time check_in_time "NOT NULL"
        time check_out_time "NOT NULL"
        text manager_id FK "NOT NULL → user"
        timestamp created_at "NOT NULL"
        timestamp updated_at "NOT NULL"
        timestamp deleted_at "NULLABLE"
    }

    suite {
        text id PK
        text title "NOT NULL"
        text description
        decimal price "NOT NULL"
        text main_image "NOT NULL"
        int capacity "NOT NULL"
        decimal area
        text establishment_id FK "NOT NULL"
        timestamp created_at "NOT NULL"
        timestamp updated_at "NOT NULL"
        timestamp deleted_at "NULLABLE"
    }

    image {
        text id PK
        text url "NOT NULL"
        text alt
        int position "NOT NULL"
        text suite_id FK "NOT NULL, CASCADE"
        timestamp created_at "NOT NULL"
    }

    amenity {
        text id PK
        text name UK "NOT NULL"
        text slug UK "NOT NULL"
        text category "NOT NULL"
        text scope "property | room | both"
        text icon
    }

    establishment_amenity {
        text establishment_id FK "PK"
        text amenity_id FK "PK"
    }

    suite_amenity {
        text suite_id FK "PK"
```

```
        text amenity_id FK "PK"
    }

    option {
        text id PK
        text name "NOT NULL"
        text slug UK "NOT NULL"
        text description
        text icon
        text pricing_model "NOT NULL"
        decimal default_price "NOT NULL"
    }

    establishment_option {
        text establishment_id FK "PK"
        text option_id FK "PK"
        decimal price "NOT NULL"
        boolean included "DEFAULT false"
    }

    booking {
        text id PK
        text reference UK "CDL-2026-XXXX"
        date check_in "NOT NULL"
        date check_out "NOT NULL"
        int guest_count "NOT NULL"
        decimal price_per_night "NOT NULL, snapshot"
        decimal total_price "NOT NULL, facture finale"
        text status "NOT NULL"
        timestamp cancelled_at
        text client_id FK "NOT NULL → user"
        text suite_id FK "NOT NULL → suite"
        timestamp created_at "NOT NULL"
        timestamp updated_at "NOT NULL"
    }

    booking_option {
        text booking_id FK "PK, CASCADE"
        text option_id FK "PK"
        int quantity "NOT NULL"
        decimal unit_price "NOT NULL, snapshot"
    }

    review {
        text id PK
        int rating "NOT NULL, 1-5"
        text comment
        boolean flagged "DEFAULT false"
        text booking_id FK "NOT NULL, UNIQUE"
        text user_id FK "NULLABLE, SET NULL"
        timestamp created_at "NOT NULL"
        timestamp updated_at "NOT NULL"
    }
```

```
    inquiry {
        text id PK
        text name "NOT NULL"
        text email "NOT NULL"
        text subject "NOT NULL, enum"
        text message "NOT NULL"
        text status "DEFAULT unread"
        text establishment_id FK "NULLABLE"
        text user_id FK "NULLABLE"
        timestamp created_at "NOT NULL"
    }
```

### 3.5 Implémentation

*Le schéma Drizzle dans `src/lib/db/schema.ts` constitue l'implémentation directe de ce MPD.*